

HisabDo AI — Day 15

Omesha — Ask HisabDo AI Query Classification & Query-to-Data Mapping

Complete detailed specification + .NET/C# reference implementation

1. Module Objective

The purpose of this module is to understand a user's financial question, classify it into a supported Ask HisabDo AI category, identify the verified HisabDo data required to answer it, and route the request to the correct backend/AI flow. The classifier is not the financial calculation engine. Financial numbers must come from verified HisabDo backend results.

2. Omesha — Exact Task Coverage

1. Define Ask HisabDo question categories.
2. Define financial query classification logic.
3. Identify what data is required for each question type.
4. Define query-to-data mapping.
5. Define intent detection methodology.
6. Handle questions about income and expenses.
7. Handle spending/category questions.
8. Handle budget questions.
9. Handle financial health questions.
10. Handle anomaly questions.
11. Handle cash-flow forecast questions.
12. Handle budget recommendation questions.
13. Handle unsupported or ambiguous questions.
14. Validate query classification using sample questions.
15. Document assumptions and limitations.

3. Supported Ask HisabDo Question Categories

Category	Meaning	Primary Data
Income & Expense	Questions about income, salary, earnings, total expenses, spending	Total income, total expenses, spending
Spending / Category	Questions about top categories, category-wise spending, where money is going	Expenses by category, amount spent, comparison period
Budget	Questions about budget status, utilization, over/under budget, remaining budget	Budget category, existing budget, target period
Financial Health	Questions about financial health score and its verified factors.	Verified health result, saving rate, expense ratio, budget utilization
Anomaly	Questions about unusual expenses, spikes, unusual categories, or duplicated transactions	Unusual expenses, duplicated transactions and historical baseline
Cash-Flow Forecast	Questions about expected future income, expenses, savings, or closing balance	Cash flow history where available, monthly aggregates, opening balance
Budget Recommendation	Questions asking what budget to use, how to adjust budgets, where to reduce spending, history where needed	Existing budgets, v
Unsupported	Questions outside supported financial capabilities or requiring information not available on HisabDo.	Not applicable
Ambiguous	Questions with insufficient wording or competing intents.	Clarification only; do not guess

4. Financial Query Classification Logic

- Step 1 — Normalize the question: trim whitespace, lowercase for matching, and normalize repeated spaces.
- Step 2 — Detect strong intent signals using category-specific keywords/phrases.
- Step 3 — Give precedence to specific intents such as anomaly, forecast, recommendation, health, and budget when generic words like 'spend' also appear.
- Step 4 — Score remaining supported categories from matching terms.
- Step 5 — If there is no evidence, classify as Unsupported.
- Step 6 — If two intents have equal/competing evidence, classify as Ambiguous and request clarification.
- Step 7 — For a supported intent, return the category plus required verified data.
- Step 8 — The data service retrieves only the authenticated user's permitted data.
- Step 9 — Existing Day 10–14 calculation engines produce verified financial values.
- Step 10 — The AI explanation layer receives verified results and explains them without changing or inventing numbers.

5. Intent Detection Methodology — MVP

For the MVP, use a simple and explainable rule/keyword-based classifier. This is preferred initially because every classification can be inspected and corrected. A later version may use an ML/LLM classifier, but it must still map to the same controlled intent taxonomy and must not bypass authorization or verified backend calculations.

Recommended priority:

Anomaly → Cash-Flow Forecast → Budget Recommendation → Financial Health → Budget → Spending/Category → Income & Expense → Unsupported/Ambiguous

6. Query-to-Data Mapping

Question Type	Data Required
Income & Expense	Transactions filtered by authenticated UserId; income/expense type; amount; date; category; requested period.
Spending/Category	Expense transactions; category; amount; date; current/previous period; verified Day 11 patterns where relevant.
Budget	Expense transactions + existing budgets + budget period + verified utilization/recommendation values where applicable.
Financial Health	Verified Day 10 health result; factor scores and supporting verified metrics.
Anomaly	Verified Day 12 anomaly result; anomaly type; amount; category; baseline; severity; timestamp/date where available.
Cash-Flow Forecast	Verified Day 13 forecast; 3–6 months history where available; monthly income/expense; opening balance if trusted.
Budget Recommendation	Verified Day 14 recommendation; historical category average; existing budget; recommended budget; variance/percentage.

7. Handling Rules by Question Type

- Income & Expense:** Retrieve verified transactions for the requested period. Calculate totals in the backend only. If the user asks 'how much did I earn last month?', route to income data and the relevant date range.
- Spending/Category:** Use expense-only verified data. For top categories, use Day 11 verified category results. For change questions, use current and previous periods.
- Budget:** Use existing budget records and verified expense totals/utilization. Never invent a budget if no budget exists.
- Financial Health:** Return the verified Day 10 score/factors. AI explains the score; it does not recompute or replace the score.
- Anomaly:** Use Day 12 verified anomaly results. Do not turn an anomaly into an accusation of fraud/theft without trusted evidence.

Cash-Flow Forecast: Use Day 13 verified forecast. Explain confidence and limitations. Do not make a new prediction inside the LLM.

Budget Recommendation: Use Day 14 verified recommendation. Explain the recommendation and its basis; do not create a different amount.

8. Unsupported & Ambiguous Questions

Unsupported examples: stock-market advice, external bank balances not connected to HisabDo, medical/legal advice, or questions requiring data HisabDo does not provide.

Ambiguous example: 'How am I doing?' — could mean financial health, spending, or budget. Ask a short clarification instead of guessing.

Missing data example: 'What is my budget utilization?' when no budget exists. State that the budget data is unavailable and do not fabricate a percentage.

Out-of-scope example: 'What will the dollar rate be next month?' should not be answered as a HisabDo forecast unless a separate verified external-data capability exists.

9. Sample Classification Validation

Sample Question	Expected Category
How much did I earn this month?	Income & Expense
How much did I spend last month?	Income & Expense
Where is most of my money going?	Spending / Category
Which category did I spend the most on?	Spending / Category
Am I over my food budget?	Budget
How much of my budget have I used?	Budget
What is my financial health score?	Financial Health
Why is my financial health score low?	Financial Health
Did I have any unusual expenses?	Anomaly
Was there a sudden spending spike?	Anomaly
How much will I spend next month?	Cash-Flow Forecast
What will my closing balance be?	Cash-Flow Forecast
What budget do you recommend for food?	Budget Recommendation
Where should I reduce my spending?	Budget Recommendation
Tell me something about my finances.	Ambiguous
What is today's weather?	Unsupported

10. Classification Result Contract

```
{
  "category": "CashFlowForecast",
  "confidence": 0.95,
  "requiredData": [
    "Authenticated UserId",
    "3-6 months historical transactions",
    "Monthly income",
    "Monthly expenses",
    "Opening balance if available",
    "Verified forecast result"
  ],
  "reason": "Detected supported intent: CashFlowForecast.",
  "missingClarification": []
}
```

11. .NET/C# Implementation

The companion C# file implements the MVP classifier, controlled categories, required-data mapping, precedence rules, unsupported handling, and ambiguous handling. It is intentionally separate from financial calculations.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;
using System.Threading;
using System.Threading.Tasks;
namespace HisabDo.AI.Day15;
/// <summary>
/// Day 15 – Ask HisabDo AI
/// Query classification and query-to-data mapping reference implementation.
/// Important: this layer identifies intent and required verified data.
/// It does NOT calculate financial values and does NOT invent answers.
/// </summary>
public enum FinancialQueryCategory
{
    IncomeExpense,
    SpendingCategory,
    Budget,
    FinancialHealth,
    Anomaly,
    CashFlowForecast,
    BudgetRecommendation,
    Unsupported,
    Ambiguous
}
public sealed record ClassificationResult(
    FinancialQueryCategory Category,
    double Confidence,
    IReadOnlyList<string> RequiredData,
    string Reason,
    IReadOnlyList<string> MissingClarification
);
public interface IFinancialQueryClassifier
{
    ClassificationResult Classify(string question);
}
public sealed class AskHisabDoQueryClassifier : IFinancialQueryClassifier
{
    private static readonly string[] IncomeExpense =
    {
        "income", "salary", "earned", "earning", "expense", "expenses",
        "spent", "spend", "cost", "how much did i spend", "how much did i earn"
    };
    private static readonly string[] Spending =
    {
        "spending", "spend", "category", "food", "transport", "shopping",
        "where is my money going", "highest spending", "top spending",
        "which category", "category-wise"
    };
    private static readonly string[] Budget =
    {
        "budget", "budget used", "budget utilization", "over budget",
        "remaining budget", "under budget", "budget limit"
    };
    private static readonly string[] Health =
    {
        "financial health", "health score", "financial score",
        "saving rate", "expense ratio", "cash flow health"
    };
    private static readonly string[] Anomaly =
    {
        "anomaly", "unusual", "unusually", "strange spending",
        "spike", "sudden spending", "duplicate transaction",
        "duplicate expense", "abnormal"
    };
    private static readonly string[] Forecast =
    {
        "forecast", "predict", "prediction", "future cash flow",
        "next month", "next month cash", "expected income",
        "expected expense", "expected savings", "closing balance"
    };
    private static readonly string[] Recommendation =
    {
        "recommend", "recommendation", "suggest", "suggestion",
        "what should i do", "how should i budget", "recommended budget",
        "save more", "where should i cut", "budget adjustment"
    }
}
```

```

};
public ClassificationResult Classify(string question)
{
    if (string.IsNullOrEmpty(question))
    {
        return new(
            FinancialQueryCategory.Ambiguous,
            0,
            Array.Empty<string>(),
            "Question is empty.",
            new[] { "Please provide a financial question." });
    }
    var q = Normalize(question);
    var scores = new Dictionary<FinancialQueryCategory, int>
    {
        [FinancialQueryCategory.IncomeExpense] = Score(q, IncomeExpense),
        [FinancialQueryCategory.SpendingCategory] = Score(q, Spending),
        [FinancialQueryCategory.Budget] = Score(q, Budget),
        [FinancialQueryCategory.FinancialHealth] = Score(q, Health),
        [FinancialQueryCategory.Anomaly] = Score(q, Anomaly),
        [FinancialQueryCategory.CashFlowForecast] = Score(q, Forecast),
        [FinancialQueryCategory.BudgetRecommendation] = Score(q, Recommendation)
    };
    // Specific intent signals take priority over generic words such as "spend".
    if (ContainsAny(q, Anomaly)) return Build(FinancialQueryCategory.Anomaly, scores);
    if (ContainsAny(q, Forecast)) return Build(FinancialQueryCategory.CashFlowForecast, scores);
    if (ContainsAny(q, Recommendation)) return Build(FinancialQueryCategory.BudgetRecommendation, scores);
    if (ContainsAny(q, Health)) return Build(FinancialQueryCategory.FinancialHealth, scores);
    if (ContainsAny(q, Budget)) return Build(FinancialQueryCategory.Budget, scores);
    var best = scores.OrderByDescending(x => x.Value).First();
    var second = scores.OrderByDescending(x => x.Value).Skip(1).First();
    if (best.Value == 0)
    {
        return new(
            FinancialQueryCategory.Unsupported,
            0,
            Array.Empty<string>(),
            "No supported financial intent was detected.",
            Array.Empty<string>());
    }
    if (best.Value == second.Value)
    {
        return new(
            FinancialQueryCategory.Ambiguous,
            0.5,
            Array.Empty<string>(),
            "Multiple financial intents have equal evidence.",
            new[] { "Ask whether the user wants spending details, budget status, forecast, health, anomaly, or recommendation." });
    }
    return Build(best.Key, scores);
}
private static ClassificationResult Build(
    FinancialQueryCategory category,
    Dictionary<FinancialQueryCategory, int> scores)
{
    var required = RequiredDataFor(category);
    var max = Math.Max(1, scores.Values.Max());
    var confidence = Math.Min(0.99, scores[category] / (double)max);
    return new(
        category,
        confidence,
        required,
        $"Detected supported intent: {category}.",
        Array.Empty<string>());
}
private static IReadOnlyList<string> RequiredDataFor(FinancialQueryCategory category) =>
    category switch
    {
        FinancialQueryCategory.IncomeExpense => new[]
        {
            "Authenticated UserId", "Transactions", "TransactionType",
            "Amount", "Date", "Category"
        },
        FinancialQueryCategory.SpendingCategory => new[]
        {
            "Authenticated UserId", "Expense Transactions",
            "Amount", "Category", "Date", "Current and previous period where comparison is requested"
        },
        FinancialQueryCategory.Budget => new[]
        {
            "Authenticated UserId", "Expense Transactions",
            "Category", "Amount", "Existing Budgets", "Budget Period"
        },
    },

```

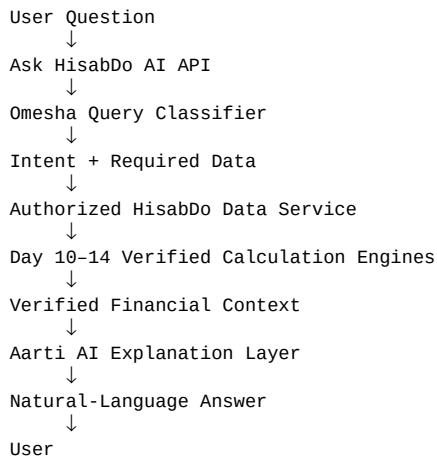
```

FinancialQueryCategory.FinancialHealth => new[]
{
    "Authenticated UserId", "Verified Financial Health Result",
    "Saving Rate", "Expense Ratio", "Budget Utilization",
    "Cash Flow", "Debt/Udhaar Result", "Expense Growth", "0-100 Score"
},
FinancialQueryCategory.Anomaly => new[]
{
    "Authenticated UserId", "Expense Transactions",
    "Verified Anomaly Results", "Historical Baseline"
},
FinancialQueryCategory.CashFlowForecast => new[]
{
    "Authenticated UserId", "3-6 Months Historical Transactions where available",
    "Monthly Income", "Monthly Expenses", "Opening Balance if available",
    "Verified Forecast Result"
},
FinancialQueryCategory.BudgetRecommendation => new[]
{
    "Authenticated UserId", "3-6 Months Expense History where available",
    "Category Spending", "Existing Budgets where available",
    "Verified Budget Recommendation Result"
},
_ => Array.Empty<string>()
};

private static int Score(string q, IEnumerable<string> keywords) =>
    keywords.Count(k => q.Contains(k, StringComparison.OrdinalIgnoreCase));
private static bool ContainsAny(string q, IEnumerable<string> keywords) =>

```

12. Integration Architecture



13. Security & Data Isolation

Never classify or answer using another user's records.

UserId must come from the authenticated session/token, not from a user-supplied prompt.

The query classifier should never be trusted as an authorization layer.

Backend services must enforce user-level authorization before retrieving financial data.

Only the minimum verified data required for the detected intent should be passed to the explanation layer.

LLM output must not override backend values.

14. Assumptions

The MVP uses controlled financial categories defined by the Day 15 Ask HisabDo scope.

Day 10–14 engines remain the source of truth for health, spending patterns, anomalies, forecasts, and budget recommendations.

Historical periods and budget periods are resolved by backend services.

Keyword matching is an MVP approach and may require refinement for Urdu/Roman Urdu and natural paraphrases.

Real production integration requires mapping these contracts to the actual HisabDo .NET entities, repositories, authentication, and endpoints.

15. Limitations & Future Improvements

Keyword matching can miss synonyms and multilingual phrasing.

Some questions contain multiple intents and need clarification or a multi-intent workflow.

Confidence in this MVP is a heuristic signal, not a statistically calibrated probability.

Future versions can add a trained classifier or LLM-based intent detection, multilingual support, entity/date/amount extraction, multi-intent decomposition, and evaluation dashboards.

Any future AI classifier must still output only a controlled intent and required-data contract; it must not invent financial facts.

16. Acceptance Checklist

- All supported categories are defined.
- Classification logic is documented and deterministic for MVP.

- Required data is mapped for every category.
- Query-to-data routing is defined.
- Income/expense, spending, budget, health, anomaly, forecast, and recommendation questions are covered.
- Unsupported and ambiguous cases are handled safely.
- Sample questions are validated against expected categories.
- User authorization is enforced outside the classifier.
- Financial calculations remain in verified backend engines.
- AI explanation receives verified results only.
- Assumptions and limitations are documented.
- C# reference implementation is ready for integration.

17. Final Definition

Omesha's Day 15 module converts a user's natural-language financial question into a controlled Ask HisabDo intent and a precise query-to-data requirement. It ensures that the correct HisabDo backend data and verified Day 10–14 results are used before Aarti's AI explanation layer generates the final response. The classifier identifies what the user is asking; it does not invent, calculate, authorize, or replace verified financial data.